

# One-dimensional arrays

CSC103 Programming Fundamentals / Lecture 20

**Dr. Muhammad Farhan**

Associate Professor of Computer Science  
Department of Computer Science  
COMSATS University Islamabad  
Sahiwal Campus

Fall 2026

# The problem for this lecture

Find the maximum in  $\{-4,-9,-2\}$ . State the policy for an empty array.

Require a nonempty array before accessing `values[0]`.

# Learning objectives

- Create arrays with fixed or runtime sizes
- Traverse elements safely
- Compute an aggregate and explain bounds errors

CLO-2

Array declaration and creation

Indexes and length

Array traversal

Runtime size and aggregation

# Array declaration and creation

- An array groups indexed elements of one component type.
- An array reference and the array object are different things.
- The array length stays fixed after construction.

## Example

```
int[] marks = {60, 75, 90};  
int[] counts = new int[4];
```

# Indexes and length

- Indexes start at zero and end at length-1.
- length is an array property, without parentheses.
- An invalid index raises `ArrayIndexOutOfBoundsException`.

## Example

```
marks[0]      // 60
marks.length  // 3
marks[3]      // invalid
```

# Array traversal

- An indexed loop supports reading and updating elements.
- An enhanced for loop conveniently reads each value.
- Keep the condition  $i < \text{array.length}$ .

## Example

```
int total = 0;
for (int value : marks) {
    total += value;
}
```

# Runtime size and aggregation

- Read or compute a size before new int[size].
- Reject negative sizes and unreasonable requests.
- A mean or maximum needs an explicit empty-array policy.

## Example

```
int size = 5;  
int[] data = new int[size];  
// data has indexes 0..4
```

# The initial maximum must come from the domain

- Zero is an incorrect default maximum when all values can be negative.
- For a nonempty array, the first element supplies a valid candidate.
- Each comparison then maintains the maximum of the processed prefix.

## Example

Input: {-4,-9,-2}

Start at -4. Compare -9, then -2.

Final maximum: -2

# A traversal bound is part of correctness

- For an array of length  $n$ , valid indexes satisfy  $0 \leq i < n$ .
- Starting at 1 is appropriate after using index 0 as the initial candidate.
- A length-zero input has no first element, so check it before initialisation.

## Example

```
int maximum = values[0];  
for (int i=1; i<values.length; i++) ...
```

# An array stores indexed elements



## What to notice

A length-4 array has indexes 0 through 3. Index 4 is outside the array.

# Key distinctions

| Index                      | 0   | 1   | 2   |
|----------------------------|-----|-----|-----|
| Element                    | -4  | -9  | -2  |
| Valid for length 3?        | Yes | Yes | Yes |
| Candidate after processing | -4  | -4  | -2  |

These distinctions determine which operation or control structure fits the problem.

# First example: execution

```
int[] marks = {60, 75, 90};  
int total = 0;  
for (int i = 0; i < marks.length; i++) {  
    total += marks[i];  
}  
System.out.println((double) total / marks.length);
```

# First example: result

75.0

The running sums are 60, 135 and 225.

The loop uses indexes 0, 1 and 2.

Array traversal

Runtime size and aggregation

# Guided practice

Find the maximum in  $\{-4,-9,-2\}$ . State the policy for an empty array.

Attempt the solution before continuing.  
Use the definitions and examples from this lecture.

# Solution strategy

- Require a nonempty array before accessing values[0].
- Initialise the maximum from the first element.
- Visit each remaining element and replace the candidate only when the new value is greater.

The next program uses fixed inputs so its result can be reproduced.

# Complete solution (1/2)

```
public class Solution20 {  
    public static void main(String[] args) throws Exception {  
        int[] values = {-4, -9, -2};  
        System.out.println(maximum(values));  
    }  
    static int maximum(int[] values) {  
        if (values == null || values.length == 0) {  
            throw new IllegalArgumentException("nonempty array required");  
        }  
    }  
}
```

## Complete solution (2/2)

```
    }  
    int maximum = values[0];  
    for (int i = 1; i < values.length; i++) {  
        if (values[i] > maximum) maximum = values[i];  
    }  
    return maximum;  
}  
}
```

# Execution trace

| Step           | Compared value | Previous maximum | New maximum |
|----------------|----------------|------------------|-------------|
| Initialisation | values[0] = -4 | None             | -4          |
| i = 1          | -9             | -4               | -4          |
| i = 2          | -2             | -4               | -2          |

Follow the changing state in execution order.

# Solution result

-2

Why this result follows  
Visit each remaining element and  
replace the candidate only when the  
new value is greater.

# A common incorrect approach

```
for(int i=0; i<=a.length; i++) {  
    System.out.println(a[i]);  
}
```

## Example

Use `i<a.length`. The last valid index is `length-1`.

# Boundary and failure checks

Check the stated input assumptions.  
Read five temperatures into an array. Print the mean and number above the mean.

Initialise max from the first element, then compare remaining values.  
Result: -2. Reject empty input or return an explicitly designed optional result.

# Check your understanding

What values does `new int[3]` contain? Does changing the enhanced-for variable update the array?

Write a prediction and explain the rule that supports it.

# Answer and explanation

Three zeros. No, for a primitive component it only changes the loop variable.

Use  $i < a.length$ . The last valid index is  $length-1$ .

# Compare cases and predict outcomes

| Index     | Element    | Running sum after visit |
|-----------|------------|-------------------------|
| 0         | 12         | 12                      |
| 1         | 7          | 19                      |
| 2, then 3 | 19, then 4 | 38, then 42             |

## Discuss

Explain each expected result before running the example. Which case would reveal a wrong assumption?

# Apply the idea

## Independent practice

Write a loop to count the values above 10 in  $\{12, 7, 19, 4\}$ . Give the count and identify the indexes that qualify.

1. Work through the input by hand.
2. Write or correct the program.
3. Justify the expected result.

# Solution and reasoning

```
int[] values = {12, 7, 19, 4};  
int count = 0;  
for (int value : values) {  
    if (value > 10) count++;  
}  
System.out.println(count);
```

- Values 12 and 19 qualify, at indexes 0 and 2.
- The output is 2.
- An enhanced for loop is sufficient because no element needs modification.

# Linear search and the first match

- Compare the target with successive elements until the first match.
- Return its index, not the element value. Use -1 when no match exists.
- Linear search works without sorting; binary search has a sorted-input precondition.

Source: [supplied ISB campus slides](#). See [speaker notes](#) and [ISB\\_Source\\_Map.md](#).

# Worked problem: Linear search and the first match

## Task

Search  $\{5,8,12,3,9\}$  for 12 and for 7, using the source search contract.

Input: Values are fixed in the program for a reproducible demonstration.

## Before running

Predict the output and identify the variables that change.

# Complete ISB example (1/1)

```
public class IsbLecture20 {  
    public static void main(String[] args) throws Exception {  
        int[] values = {5, 8, 12, 3, 9};  
        System.out.println(linearSearch(values, 12));  
        System.out.println(linearSearch(values, 7));  
    }  
    static int linearSearch(int[] values, int key) {  
        for (int i = 0; i < values.length; i++) {  
            if (values[i] == key) return i;  
        }  
        return -1;  
    }  
}
```

# Worked trace and expected result

| Index visited | Value | Searching for 12 |
|---------------|-------|------------------|
| 0             | 5     | Continue         |
| 1             | 8     | Continue         |
| 2             | 12    | Return 2         |

## Expected console output

```
2  
-1
```

# Extend the ISB example

## Practice

Predict searching  $\{12,5,12\}$  for 12 and searching an empty array.

## Explain your reasoning

The first returns 0; the empty array returns -1 without entering the loop.

# Lecture summary

- arrays with fixed or runtime sizes
- Traverse elements safely
- an aggregate and explain bounds errors
- Require a nonempty array before accessing values[0].
- Initialise the maximum from the first element.
- Visit each remaining element and replace the candidate only when the new value is greater.

# After-class practice

Read five temperatures into an array. Print the mean and number above the mean.

Syllabus reading

Deitel Ch 7

Next lecture: Arrays and method arguments